Lab 7 — Fuzz Testing (Shopping Cart)

Implementation notes and validation record

This document summarizes the repository inspection and the prepared implementation. It is not the screenshot evidence PDF requested by the lab handout; screenshots from actual runs on the student machine still need to be inserted.

1. Source and repository mapping

The supplied handout defines a three-function fuzzing exercise. The assigned repository uses the folder **Lab 7** and names its files **shopping_cart.py** and **test_shopping_cart.py** (rather than the generic processor.py and test_processor.py examples). The functions are cart_total, promo_percent, and delivery_zone.

2. Bugs observed in the starter implementation

Function	Weakness in starter code
cart_total	Assumes comma/colon/x delimiters always exist and occur exactly where expected; unpacking or integer conversion fails on malformed values. Does not reject empty names, negative values, or empty entries explicitly.
promo_percent	Splits on OFF and indexes the second piece; missing marker causes IndexError. Does not validate digits, empty suffix, or percentage range.
delivery_zone	Indexes the first pincode character and looks it up directly; empty input can raise IndexError, unknown leading digit raises KeyError, and length/character validity are not checked.

3. Fixed input contract

Cart: one or more comma-separated name:pricexqty entries. Names cannot be empty; price and quantity must be non-negative decimal integers. Invalid text raises ValueError; non-string input raises TypeError.

Promo: OFF followed by digits, case-insensitive, with a value from 0 through 100 inclusive. Invalid text raises ValueError; non-string input raises TypeError.

Pincode: exactly six ASCII digits, with first digit 1–8 supported by the starter ZONES map. Invalid text raises ValueError; non-string input raises TypeError.

4. Hypothesis test strategy

The completed test module uses st.one_of(st.text(), st.none()) for each of the three API inputs. It checks that text inputs either return a valid result or are rejected with ValueError, and None is rejected with TypeError. Additional generated properties verify cart totals against a computed sum, promo values from 0 to 100, and pincode-zone mappings for supported first digits. A concrete known-example test covers pen:10x2,book:250x1; OFF20; and 560085.

5. Local validation performed in this environment

Python compilation of both prepared files succeeded. Deterministic smoke checks covered valid cart, promo and pincode examples, as well as malformed and non-string inputs; all checks passed. Pytest is present, but Hypothesis is not installed in this execution environment. Installing it failed because the environment could not resolve/reach the package index. Therefore the Hypothesis pytest suite has NOT been executed here.

6. Remaining actions on the student machine

1. Keep or restore the repository’s original starter files and run the original interactive app once. Capture the actual terminal output (the PDF requires the initial buggy run).

2. Complete/run the fuzz tests against the original buggy implementation and capture a genuine failing-test screenshot.

3. Copy the prepared fixed shopping_cart.py and test_shopping_cart.py into the repository Lab 7 folder.

4. Install dependencies with python -m pip install hypothesis pytest.

5. Run python shopping_cart.py and capture the fixed app output. Use example inputs pen:10x2,book:250x1; OFF20; 560085.

6. Run python -m pytest -v test_shopping_cart.py and capture the passing test output.

7. Combine the four genuine screenshots into the submission evidence PDF.

7. Example expected results (for reference)

```
cart_total("pen:10x2,book:250x1")  -> 270
promo_percent("OFF20")                -> 20
delivery_zone("560085")                -> South
```

The implementation files are the authoritative code deliverables. This notes PDF is explanatory only and must not be submitted as if it contained the required terminal screenshots.